

LAPORAN PRAKTIKUM
POSTTEST (6)
ALGORITMA PEMROGRAMAN LANJUT

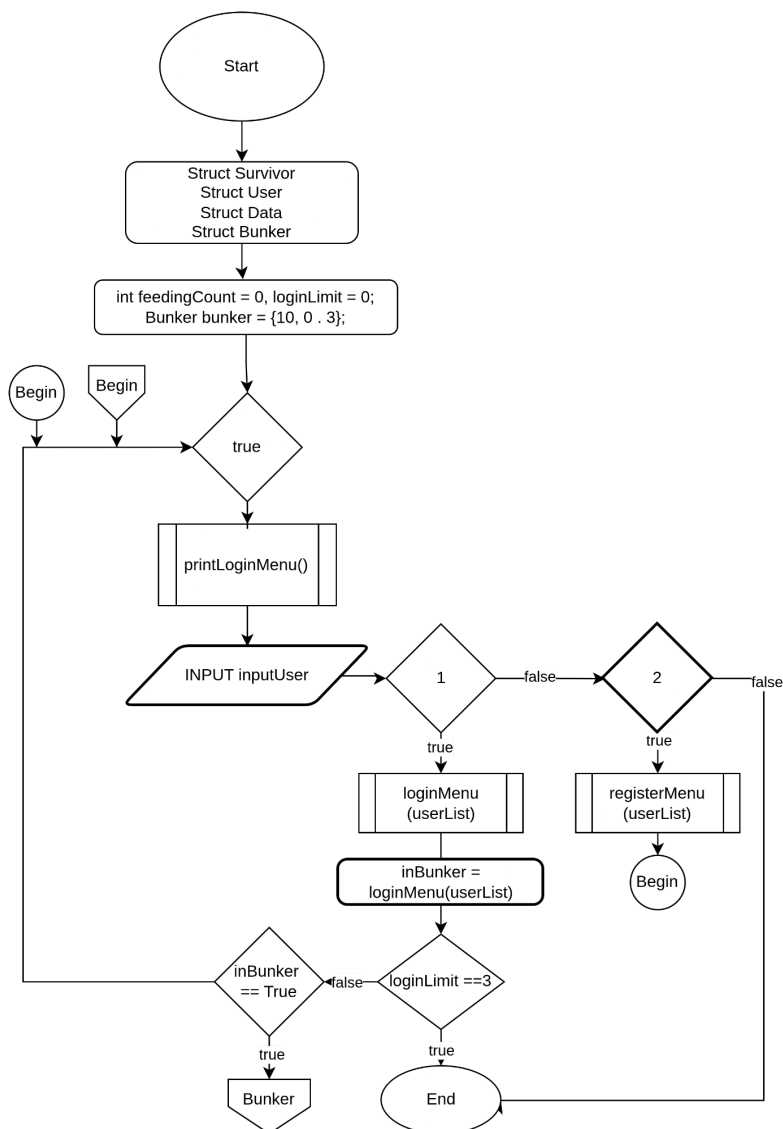


Disusun oleh:
Akbar Rachim (2509106028)
Kelas (A'25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Flowchart

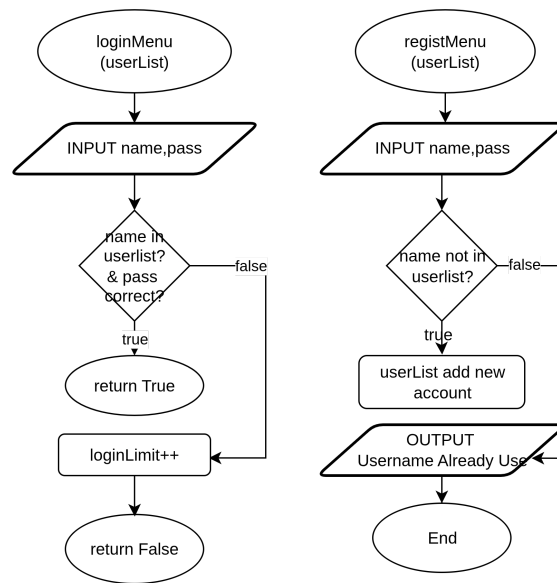
1.1. Menu Awal



Gambar 1.1. Flowchart Menu Awal

Pada menu awal ini program dimulai, user dapat memilih terlebih dahulu ingin login atau register terlebih dahulu, jika user gagal login selama tiga kali makan akan langsung dikeluarkan dari program, namun jika user berhasil login user akan dibawa masuk ke menu bunker.

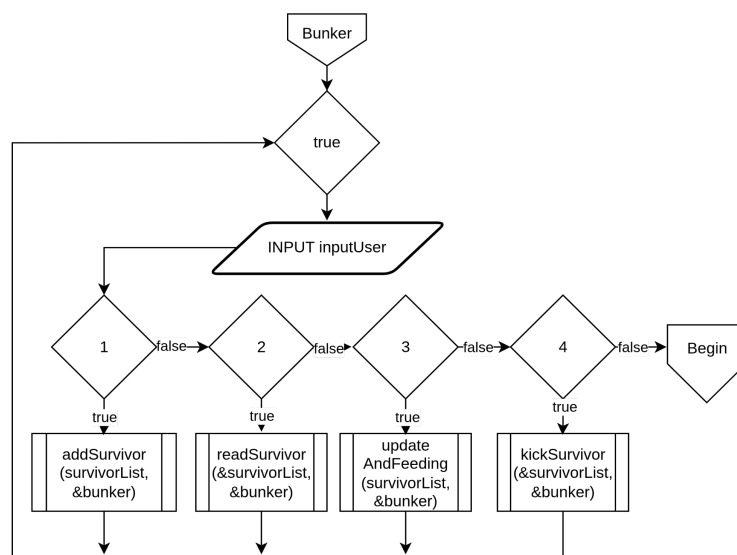
1.2. Fungsi dan Prosedur Login dan Register



Gambar 1.2 Flowchart Login dan Register

Pada fungsi login, pengguna diminta memasukkan username dan password. Jika data sesuai, pengguna akan diarahkan ke menu bunker. Begitu pula pada proses registrasi, pengguna diminta mengisi username dan password, jika data valid, registrasi dinyatakan berhasil.

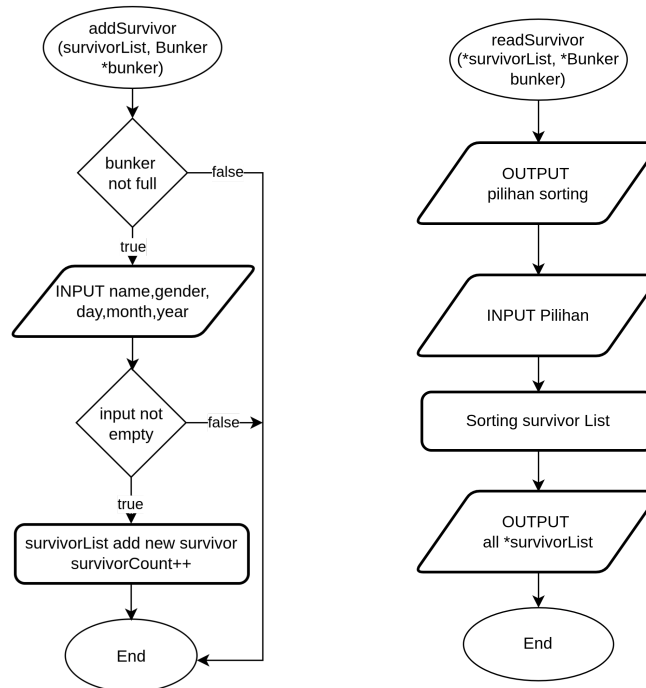
1.3. Menu Bunker



Gambar 1.3. Flowchart Menu Bunker

Pada menu bunker user dapat memilih untuk menambah, melihat, memperbarui dan menghapus survivor dari bunker, jika user tidak memilih dari keempatnya user akan di kembalikan ke menu awal.

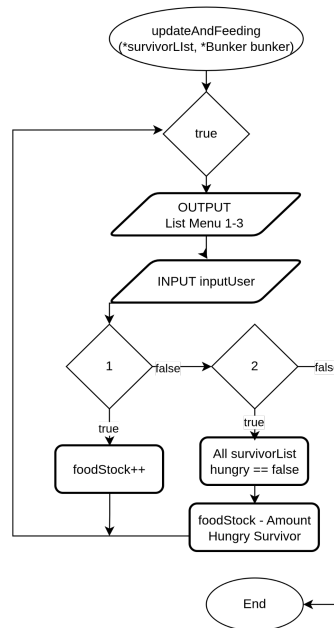
1.4. Prosedur Tambah dan Lihat List Survivor



Gambar 1.4 Flowchart Prosedur Menambah dan Melihat Survivor

Pada prosedur ini, pengguna dapat menambahkan penyintas (*survivor*) baru ke dalam bunker dengan mengisi data nama, gender, dan tanggal masuk. Jika terdapat input yang kosong, penambahan akan gagal. Selain itu, pada prosedur **readSurvivor**, pengguna dapat melihat daftar penyintas dengan fitur pengurutan (*sorting*) berdasarkan nama atau tahun secara *ascending* maupun *descending*.

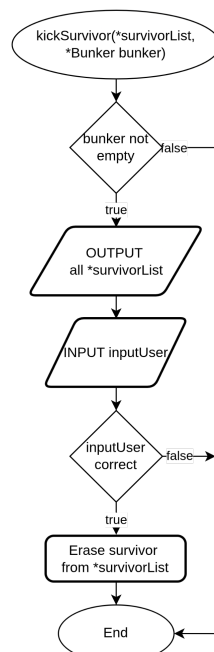
1.5. Prosedur Cari dan Beri Makan Survivor



Gambar 1.5. Flowchart Prosedur Cari Makan dan Beri Makan

Dalam prosedur ini, pengguna dapat mencari makanan untuk menambah persediaan di bunker. Jika stok mencukupi, pengguna dapat memberi makan seluruh penghuni untuk menghilangkan status lapar mereka.

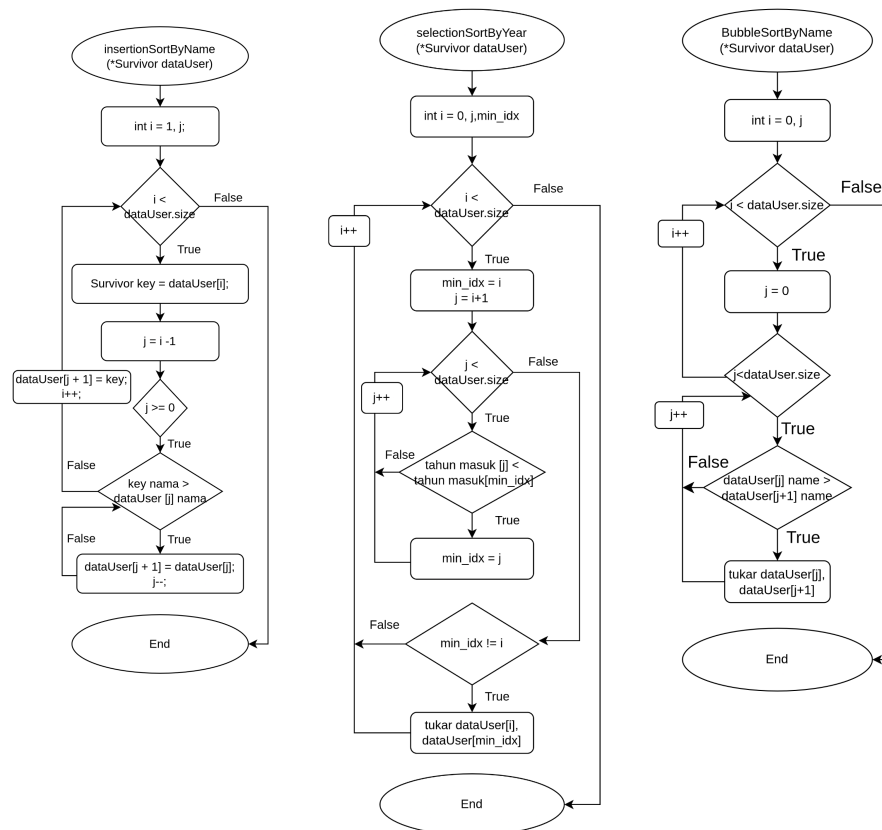
1.6. Prosedur Tendang Survivor



Gambar 1.7 Flowchart Tendang Survivor

Pada prosedur tendang survivor seperti namanya user dapat melakukan penghapusan survivor dari list survivor dengan menendang keluar survivor tersebut yang dipilih.

1.7. Prosedur Sorting List Survivor

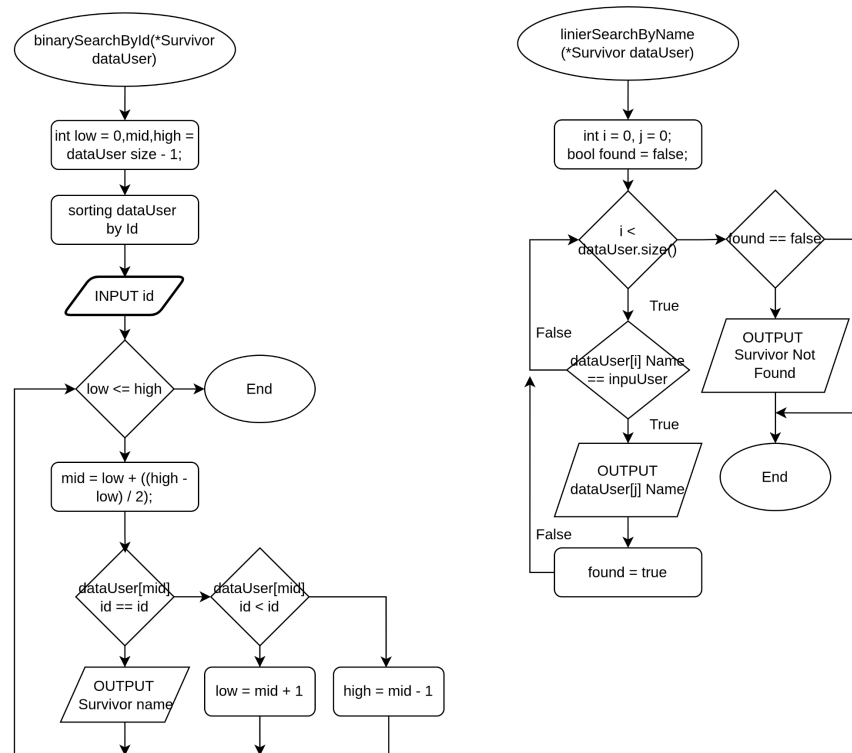


Gambar 1.7. Flowchart Prosedur Sorting

Sebagaimana ditunjukkan pada Gambar 1.7, prosedur pengurutan (*sorting*) menggunakan tiga metode berbeda, yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*. Masing-masing metode memiliki tujuan sebagai berikut:

- **Insertion Sort (By Name):** Mengurutkan nama penyintas dalam data vektor secara *descending* (Z-A).
- **Bubble Sort (By Name):** Mengurutkan nama penyintas dalam data vektor secara *ascending* (A-Z).
- **Selection Sort (By Year):** Mengurutkan tahun masuk penyintas dalam data vektor secara *ascending* (tahun terkecil ke terbesar).

1.8. Prosedur Searching



Gambar 1.8. Flowchart Prosedur Searching

Sebagaimana ditunjukkan pada Gambar 1.8, prosedur pencarian (*searching*) menggunakan dua metode berbeda, yaitu *Binary Search*, dan *Linier Search*. Masing-masing metode memiliki tujuan sebagai berikut:

- **Linier Search (By Name):** Mencari Penyintas berdasarkan nama dalam data vektor.
- **Bubble Sort (By Id):** Mencari Penyintas berdasarkan Id dalam data vektor.

2. Deskripsi Singkat Program

2.1. Sistem Keamanan & Akun

Program memiliki sistem login dengan dua level akses: **Admin** dan **User**.

- **Registrasi:** Pengguna baru dapat mendaftar jika slot akun maksimal masih tersedia.

- **Autentikasi:** Login dibatasi maksimal 3 kali percobaan kegagalan sebelum program otomatis tertutup.
- **Hak Akses:** Hanya Admin yang dapat menambah, memberi makan, atau mengeluarkan survivor. User biasa hanya diberikan izin untuk melihat daftar survivor.

2.2. Manajemen Data (Structs)

Program menggunakan struktur data (struct) yang saling berhubungan untuk mengorganisir informasi:

- **User:** Menyimpan username, password, dan tipe akun.
- **Date:** Menyimpan informasi tanggal (hari, bulan, tahun).
- **Survivor:** Menyimpan identitas survivor (nama, gender), status kesehatan (lapar/hidup), dan tanggal masuk ke bunker.
- **Bunker :** Menyimpan informasi stok makanan, kapasitas bunker, dan jumlah survivor yang ada.

2.3. Operasi Manajemen Bunker (CRUD)

Di dalam menu utama bunker, admin dapat melakukan pengelolaan logistik:

- **Add Survivor:** Menambah orang baru ke dalam bunker dengan batasan maksimal.
- **Read List:** Menampilkan daftar penghuni bunker beserta status rasa lapar mereka dan sisa stok makanan dengan sorting ataupun dapat melakukan searching.
- **Update (Logistik):** Fitur untuk mencari makanan (menambah stok) dan memberi makan survivor yang lapar. Memberi makan akan mengubah status hungry dari true menjadi false.
- **Kick Survivor:** Menghapus data survivor dari daftar dengan sistem penggeseran indeks array.

2.4. Efisiensi Memori

Dalam mengurangi penggunaan memori dan mempercepat kecepatan program diberlakukan beberapa hal pada program yaitu penerapan penggunaan pointer sehingga terutama pada penggunaan fungsi dan prosedur tidak terus menerus membuat variabel baru yang memakan memori dan hanya dipakai sekali.

3. Source Code

3.1. Login

```
bool loginMenu(vector<User> &userList) {
    string name, pass;
    cout << "Input Username : ";
    cin.ignore(1000, '\n');
    getline(cin, name);
    cout << "Input Password : ";
    cin >> pass;
    for (auto &account : userList) {
        if (account.userName == name && account.userPass ==
pass) {
            cout << "Login Succes\n";
            typeUser = account.userType;
            return true;
        }
    }
    cout << "Login Failed\n";
    loginLimit++;
    return false;
}

void registerMenu(vector<User> &userList) {
    cout << "Input Name : ";
    cin.ignore(1000, '\n');
    getline(cin, name);
    cout << "Input Pass : ";
    cin >> pass;
    for (auto &account : userList) {
```

```

        if (name == account.userName) {
            cout << "Username Already Use\n";
            return;
        }
    }
    userList.push_back(User(name, pass, "user"));
    cout << "Register Succes\n";
}

```

3.2. Registrasi

```

void registerMenu(vector<User> &userList) {
    cout << "Input Name : ";
    cin.ignore(1000, '\n');
    getline(cin, name);
    cout << "Input Pass : ";
    cin >> pass;
    for (auto &account : userList) {
        if (name == account.userName) {
            cout << "Username Already Use\n";
            return;
        }
    }
    userList.push_back(User(name, pass, "user"));
    cout << "Register Succes\n";
}

```

3.3. Menambahkan Survivor

```

void addSurvivor(vector<Survivor> &survivorList, Bunker
*bunker, string n,
                string g, string d, string m, string y)
{
    if (bunker->survivorCount > bunker->limit - 1) {
        cout << "Bunker Full\n";
    } else {
        if (n.empty() || g.empty() || d.empty() || m.empty()

```

```

|| y.empty()) {
    cout << "Input Cannot Empty\n";
} else {
    survivorList.push_back(Survivor(idNext,n, g, d, m,
y));
    bunker->survivorCount++;
    idNext++;
    cout << "New Survivor Succes To Add\n";
}
}
}
}

```

3.4. Melihat List Survivor

```

void readSurvivor(vector<Survivor> *survivorList, Bunker
*bunker) {
    int inputUser;
    while (true) {
        cout << "1. Sorting By Name (A-Z)\n";
        cout << "2. Sorting By Name (Z-A)\n";
        cout << "3. Sorting By Year (New-O)\n";
        cout << "4. Searching By Id\n";
        cout << "5. Searching By Name\n";
        cout << "Input User : ";
        cin >> inputUser;
        if (inputUser == 1) {
            bubbleSortByName(*survivorList);
            break;
        } else if (inputUser == 2) {
            insertionSortByName(*survivorList);
            break;
        } else if (inputUser == 3) {
            selectionSortByYear(*survivorList);
            break;
        } else if (inputUser == 4) {
            binarySearchById(*survivorList);
            return;
        } else if (inputUser == 5) {
            linierSearchByName(*survivorList);
            return;
        }
    }
}

```

```

        } else {
            continue;
        }
    }
    Table t;
    t.add_row({"No", "Id", "Name", "Gender", "Status",
"Entry Year"});

t[0].format().font_style({FontStyle::bold}).font_align(F
ontAlign::center);
    if (bunker->survivorCount == 0) {
        t.add_row({"-", "Bunker Empty", "-", "-"});
    } else {
        for (int i = 0; i < bunker->survivorCount; i++) {
            t.add_row({to_string(i + 1),
to_string((*survivorList)[i].idSurvivor),
                        (*survivorList)[i].survivorName,
                        (*survivorList)[i].survivorGender,
                        (*survivorList)[i].hungry ? "Hungry" :
"Not Hungry",
                        (*survivorList)[i].dateEntry.year});
        }
    }
    cout << t << endl;

    Table info;
    info.add_row({"Food Stock", bunker->foodStock == 0
? "Empty"
:
to_string(bunker->foodStock)});
    info[0].format().font_style({FontStyle::bold});
    if (bunker->foodStock == 0)
        info[0][1].format().font_color(Color::red);
    else
        info[0][1].format().font_color(Color::green);
    cout << info << endl;
}

```

3.5. Mencari Makanan atau Memberi Makan

```
void updateAndFeeding(vector<Survivor> &survivorList,
Bunker *bunker) {
    while (true) {
        feedingCount = 0;
        cout << "-----Update & Feeding
Menu-----\n";
        cout << "1. Search Food\n";
        cout << "2. Feeding Hungry Survivor\n";
        cout << "3. Exit\n";
        cout << "Input Id Menu : ";
        cin >> inputUser;
        if (inputUser == 1) {
            cout << "Search Food.....\n";
            bunker->foodStock += 3;
            cout << "Search Food Succes\n";
            cout << "Food Stock Now : " << bunker->foodStock
<< endl;
        } else if (inputUser == 2) {
            if (bunker->survivorCount == 0) {
                cout << "Bunker Empty\n";
                continue;
            }
            for (int i = 0; i < bunker->survivorCount; i++) {
                if (bunker->foodStock != 0 &&
survivorList[i].hungry == true) {
                    survivorList[i].hungry = false;
                    feedingCount++;
                    bunker->foodStock--;
                }
            }
            cout << "Survivor Get Feed : " << feedingCount <<
endl;
            cout << "Food Stock : " << bunker->foodStock <<
endl;
        } else if (inputUser == 3) {
            cout << endl;
            break;
        }
    }
}
```

3.6. Menghapus Survivor Dari List

```
void kickSurvivor(vector<Survivor> *survivorList, Bunker
*bunker) {
    if (bunker->survivorCount == 0) {
        cout << "Bunker Empty\n";
        return;
    }
    Table t;
    t.add_row({"No", "Name"});

    t[0].format().font_style({FontStyle::bold}).font_align(F
ontAlign::center);
    for (int i = 0; i < bunker->survivorCount; i++) {
        t.add_row({to_string(i + 1),
(*survivorList)[i].survivorName});
    }
    cout << t << endl;
    cout << "Input [0] For Exit\n";
    cout << "Input Number Survivor to Kick From Bunker :
";
    cin >> inputUser;
    if (inputUser <= 0 || inputUser >
bunker->survivorCount) {
        return;
    }
    for (int i = inputUser - 1; i < bunker->survivorCount
- 1; i++) {
        (*survivorList)[i] = (*survivorList)[i + 1];
    }
    survivorList->pop_back();
    bunker->survivorCount--;
}
```

3.7. Insertion Sort Untuk Nama

```
void insertionSortByName(vector<Survivor> &dataUser) {
    int j;
    for (int i = 1; i < dataUser.size(); i++) {
        Survivor key = dataUser[i];
        j = i - 1;
```

```

        while (j >= 0 && key.survivorName >
dataUser[j].survivorName) {
            dataUser[j + 1] = dataUser[j];
            j--;
        }
        dataUser[j + 1] = key;
    }
}

```

3.8. Bubble Sort Untuk Nama

```

void bubbleSortByName(vector<Survivor> &dataUser) {
    for (int i = 0; i < dataUser.size(); i++) {
        for (int j = 0; j < dataUser.size() - 1; j++) {
            if (dataUser[j].survivorName > dataUser[j +
1].survivorName) {
                swap(dataUser[j], dataUser[j + 1]);}}}

```

3.9. Selection Sort Untuk Tahun Masuk

```

void selectionSortByYear(vector<Survivor> &dataUser) {
    int min_index;
    for (int i = 0; i < dataUser.size(); i++) {
        min_index = i;
        for (int j = i + 1; j < dataUser.size(); j++) {
            if (dataUser[j].dateEntry.year <
dataUser[min_index].dateEntry.year) {
                min_index = j;
            }
        }
        if (min_index != i) {
            swap(dataUser[i], dataUser[min_index]);}}}

```

3.10. Linier Search untuk Nama Survivor

```

void linierSearchByName(vector<Survivor> &dataUser) {
    string inputUser;
    bool found = false;
    cin.ignore(1000, '\n');
    cout << "Input Survivor Name : ";
    getline(cin, inputUser);

```

```

    for (int i = 0; i < dataUser.size(); i++) {
        if (inputUser == dataUser[i].survivorName) {
            cout << "Survivor Found : " << dataUser[i].survivorName
<< endl;
            found = true;
        } else if (found == false) {
            cout << "Survivor Not Found\n";
        }
    }
}

```

3.11. Binary Search untuk Id Survivor

```

void binarySearchById(vector<Survivor> &dataUser) {
    int inputUser;
    int low = 0, mid, high = dataUser.size() - 1;
    cout << "Input Survivor Id : ";
    // sorting by id
    for (int i = 0; i < dataUser.size(); i++) {
        for (int j = 0; j < dataUser.size() - 1; j++) {
            if (dataUser[j].idSurvivor > dataUser[j +
1].idSurvivor) {
                swap(dataUser[j], dataUser[j + 1]);
            }
        }
    }
    cin >> inputUser;
    while (low <= high) {
        mid = low + ((high - low) / 2);
        if (dataUser[mid].idSurvivor == inputUser) {
            cout << "Survivor Found : " <<
dataUser[mid].survivorName << endl;
            return;
        } else if (dataUser[mid].idSurvivor < inputUser) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    cout << "Not Found\n";
}

```


4. Hasil Output

4.1. Login

```
-----Login Menu-----  
1. Login  
2. Regist  
3. Exit  
Input Menu Id : 1  
Input Username : Akbar Rachim  
Input Password : 028  
Login Succes
```

Gambar 4.1 Hasil Output

4.2. Registrasi

```
-----Login Menu-----  
1. Login  
2. Regist  
3. Exit  
Input Menu Id : 2  
Input Name : Rudi  
Input Pass : 123
```

Gambar 4.2 Hasil Output Registrasi

4.3. Tambah Survivor

```
-----Bunker Menu-----  
1. Add Survivor  
2. Read List Survivor  
3. Update Food Or Survivor  
4. Kick Survivor  
5. Exit  
Input Menu Id : 1  
Name : Akbar Rachim  
Gender : Male  
Day : 10  
Month : 12  
Year : 2100
```

Gambar 4.3 Hasil Output Tambah Survivor

4.4. Lihat List Survivor

```
Input Menu Id : 2
1. Sorting By Name (A-Z)
2. Sorting By Name (Z-A)
3. Sorting By Year (New-0)
4. Searching By Id
5. Searching By Name
Input User : 1
+-----+
| No | Id | Name | Gender | Status | Entry Year |
+-----+
| 1 | 0 | Budi | Male | Hungry | 2003 |
+-----+
| 2 | 2 | Gea | Female | Hungry | 2001 |
+-----+
| 3 | 1 | Radi | Male | Hungry | 2002 |
+-----+
| 4 | 3 | Zea | Female | Hungry | 2004 |
+-----+
```

Gambar 4.4.1. Hasil Output

4.5. Mencari Makanan atau Memberi Makan

```
Search Food.....
Search Food Succes
Food Stock Now : 33
-----
1. Search Food
2. Feeding Hungry Survivor
3. Exit
Input Id Menu : 2
Survivor Get Feed : 1
Food Stock : 32
```

Gambar 4.5 Hasil Output

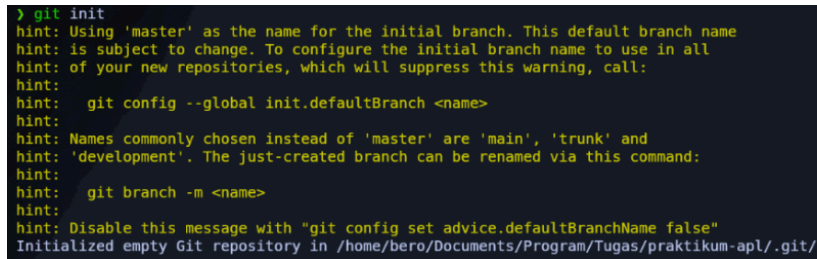
4.6. Keluarkan Survivor dari Bunker

```
+-----+
| No | Name |
+-----+
| 1 | Budi |
+-----+
| 2 | Radi |
+-----+
| 3 | Gea |
+-----+
Input [0] For Exit
Input Number Survivor to Kick From Bunker : 1
```

Gambar 4.6 Hasil Output

5. Langkah-langkah GIT

5.1. GIT Init



```
> git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/bero/Documents/Program/Tugas/praktikum-apl/.git/
```

Gambar 5.1 GIT Init

Membuat repository baru git untuk melacak setiap perubahan dalam folder sehingga setiap perubahan dapat dilihat.

5.2. GIT Remote

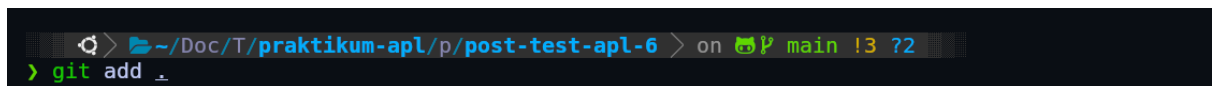


```
> git remote add origin https://github.com/cybercrym-crying/Praktikum-APL.git
```

Gambar 5.2 GIT Remote

Menghubungkan repository git lokal dengan cloud yaitu github.

5.3. GIT Add

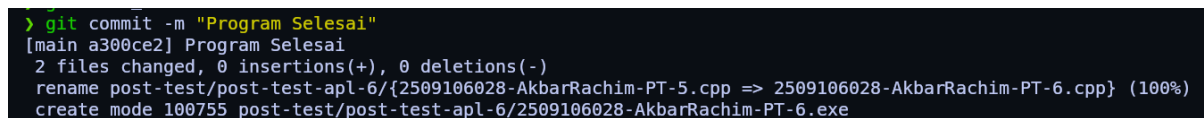


```
> git add
```

Gambar 5.3 GIT Add

Melakukan git add untuk menambahkan file dan folder post-test-apl-3 ke stage untuk di commit.

5.4. GIT Commit



```
> git commit -m "Program Selesai"
[main a300ce2] Program Selesai
2 files changed, 0 insertions(+), 0 deletions(-)
rename post-test/post-test-apl-6/{2509106028-AkbarRachim-PT-5.cpp => 2509106028-AkbarRachim-PT-6.cpp} (100%)
create mode 100755 post-test/post-test-apl-6/2509106028-AkbarRachim-PT-6.exe
```

Gambar 5.4 Git Commit

Pada gambar 5.4 git commit mulai mencatat dan menyimpan rekaman perubahan file dan folder di ke dalam git.

5.5. GIT Push

```
> git push -u origin main
Username for 'https://github.com': cybercrym-crying
Password for 'https://cybercrym-crying@github.com':
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 729.72 KiB | 4.86 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/cybercrym-crying/Praktikum-APL.git
   2b036ed..a300ce2  main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.5 GIT Push

Pada langkah git terakhir gambar 5.5 git push akan Menyimpan perubahan git (local) menuju github (cloud) secara online.